

Grandis Legacy Package Creation Rules

Anti-duplicate, clean-source, PDF-lock, and Railway/GitHub-safe packaging standard

Purpose

- Dokumen ini adalah aturan kerja saat membuat package Grandis Legacy.
- Fokus utama: tidak ada file berulang, tidak ada old/final/fix duplicate, package mudah diterapkan ke GitHub/Railway, dan jelas source-of-truth-nya.
- Aturan ini berlaku untuk Local AI, PvP Railway, Runtime Engine, Blueprint, data, dan asset package.

1. Golden Rule: No Repeated Files

- Satu fungsi = satu file aktif.
- Jangan menyimpan versi berulang seperti app_old.js, app_final.js, app_fix.js, app_new.js, app_backup.js di package final.
- Jangan menyimpan dua file yang sama dengan nama beda kecuali jelas documented sebagai source vs generated output.
- Jangan memasukkan archive lama ke dalam archive baru.
- Jika file diganti, replace file lama. Jangan tambahkan file baru dengan suffix fix kecuali sementara di workdir dan tidak dikirim.

Dilarang di Package Final	Yang Benar
app.js, app_v2.js, app_final.js	app.js saja
style.css, style_old.css, style_fix.css	style.css atau css/app.css saja sesuai struktur
index_backup.html	index.html saja
Grandis_old.zip di dalam package baru	Tidak ada nested historical zip
docs berulang tanpa fungsi	Satu release note/takeover note yang relevan

2. Source-of-Truth Rule

- Blueprint adalah implementation contract; source mengikuti Blueprint.
- Runtime Engine adalah authority untuk gameplay.
- UI/client hanya view dan input intent.
- Generated files harus jelas statusnya dan tidak dianggap source utama kecuali memang dikunci.
- Jika ada src/app.js dan public/app.js, jelaskan mana source dan mana deploy copy/bundle. Jangan biarkan keduanya divergen tanpa catatan.

Recommended Practice

- Untuk package kecil/manual: public/app.js boleh menjadi deploy file aktif, src/app.js boleh mirror jika memang dibutuhkan. Tetapi manifest/release note harus menjelaskan hubungan keduanya.
- Untuk package serius: satu source utama, satu build output, dan build output bisa diregenerate.

3. File Naming Rule

- Nama package publik harus bersih dan tidak terlalu internal.
- Local AI public-friendly naming: Grandis_Legacy_Local_AI_vX.YY_YYYY-MM-DD.zip.
- PvP public-friendly naming: Grandis_Legacy_PvP_Railway_vX.YY_YYYY-MM-DD.zip atau nama scope pendek yang jelas.
- Hindari nama panjang berisi semua detail bugfix untuk package yang akan sering diupload user.
- Detail perubahan masuk ke docs/release note, bukan semua dijejalkan ke filename.

Jenis	Contoh Baik	Catatan
Local AI	Grandis_Legacy_Local_AI_v3.20_2026-07-01.zip	Tidak perlu nama internal fix.
PvP reset	Grandis_Legacy_PvP_Railway_v1.0_2026-07-01.zip	Bersih untuk GitHub/Railway.
Patch internal	Grandis_Legacy_PvP_Railway_v1.1_2026-07-02.zip	Detail changelog di docs.

4. Package Root Layout Rule

- File yang dibutuhkan Railway harus ada di root package/repo.
- package.json dan server.js harus di root untuk Railway Node deploy.
- public/ harus berada di root.
- docs/ boleh berada di root.
- FILE_MANIFEST_SHA256.csv harus di root.
- Jangan membuat package yang setelah extract masih punya folder wrapper ganda sehingga user salah upload subfolder.

Path	Required for PvP Railway
package.json	Yes, root
server.js	Yes, root
public/index.html	Yes
public/css/app.css or public/style.css	Yes, one active style path only
public/js/app.bundle.js or public/app.js	Yes, one active app path only
public/assets/	Yes, WebP-only for PvP asset deploy
docs/	Recommended
FILE_MANIFEST_SHA256.csv	Required for audit

5. Asset Rules

- PvP package: no JPG/JPEG files. Use WebP only for card art and thumbnails.
- Reason: PvP loads assets over the internet; WebP is lighter and faster.
- Local AI may still load local files, but deployed PvP must be optimized for network loading.
- Do not include both .jpg and .webp versions of the same asset in PvP deploy.
- Use consistent paths such as public/assets/cards/ and public/assets/thumbs/.
- Remove unused assets from deploy package; do not ship full raw art archives unless required.

Allowed in PvP Deploy	Not Allowed in PvP Deploy
.webp card art	.jpg card art
.webp thumbnails	.jpeg thumbnails
small logo/background assets if used	unused raw/source art packs
manifested assets referenced by app	duplicate image formats for same card

6. Lock Files and Registry Rules

- Do not include package-lock.json if it points to an internal/private registry.
- If npm install fails on Railway because of internal registry, delete package-lock.json and use .npmrc with public npm registry.
- Safe .npmrc for Railway: registry=https://registry.npmjs.org/, fund=false, audit=false.
- package.json should be minimal and use public dependencies only.
- Do not hide environment-specific assumptions inside lockfiles.

7. Documentation Rules

- Every package should include a short docs note explaining scope, files changed, and how to apply.
- Docs should clearly state whether package is UI-only, runtime-only, asset-only, or gameplay-affecting.
- Do not claim gameplay is fixed unless tested/audited.
- Include GitHub Desktop commit summary and optional description for non-technical user.
- Include Railway validation steps when package is for PvP Railway.

Doc Type	When to Include
README.md	Full/reset package
RELEASE_NOTE.md	Versioned update package
TAKEOVER_NOTE.md	When changing workflow/source direction
FILE_MANIFEST_SHA256.csv	Every delivered package
QA_CHECKLIST.md	Recommended for runtime/gameplay changes

8. Manifest and Verification Rule

- Every package delivered to user must include FILE_MANIFEST_SHA256.csv.
- Manifest should list every file included and SHA256 hash.
- Before delivery, zip integrity should be tested.
- For JS files, run syntax check if applicable: `node --check public/app.js`, `node --check src/app.js`, or equivalent.
- For document deliverables, render and visually inspect output before delivery.
- For package deliverables, do not invent sandbox links; only link files confirmed created in `/mnt/data`.

Check	Command/Method
JS syntax	<code>node --check <file>.js</code>
Manifest	<code>find . -type f sort xargs sha256sum > FILE_MANIFEST_SHA256.csv</code>
Zip integrity	<code>unzip -tq <package>.zip</code>
Root layout	<code>ls package root</code> and confirm <code>package.json/server.js/public/docs/manifest</code>
No JPG in PvP	<code>find public/assets -iname "*.jpg" -o -iname "*.jpeg"</code> should return nothing

9. Versioning and Changelog Rule

- Increment version only when package changes are meaningful.
- Do not create many micro-patches for unclear UI guesses. If issue root is architecture, stop and rebuild clean base.
- Changelog must separate UI, runtime/gameplay, data/card definitions, assets, and deployment changes.
- Mark known limitations honestly.
- If user says “jangan update dulu”, do not create a new package; just note the rule for next package.

10. Cleanup Rule Before Shipping

1. Remove old, backup, temp, test, and duplicate files.
2. Remove nested archives unless explicitly requested.
3. Remove unused JPG/JPEG from PvP package.
4. Remove internal-only lockfiles or registry config that could break Railway.
5. Confirm active app/style paths in `index.html` match actual files.
6. Confirm package root is ready to copy directly into GitHub repo.
7. Generate fresh manifest after final cleanup.
8. Zip from inside package root so files are not nested under an unwanted folder unless intentionally documented.
9. Run zip integrity test.
10. Deliver link plus concise apply instructions.

11. GitHub/Railway Apply Instructions Template

11. Extract zip.
12. Open local GitHub repo folder.
13. Delete files that this package replaces, especially old patch-chain files.
14. Copy package contents into repo root.
15. Open GitHub Desktop and review changes.
16. Commit with provided summary.
17. Push origin/main.
18. Wait for Railway build/deploy success.
19. Open /health if available.
20. Hard refresh/Incognito to avoid browser cache.

12. Red Flags That Mean Stop and Rebuild

- UI requires manual refresh after every action.
- Patch adds more wrappers to old alpha renderer instead of replacing base.
- CSS is duplicated across index.html, style.css, and injected JS without a clear reason.
- Multiple files define the same function or renderer.
- Railway keeps showing old UI because build failed but app still serves previous deploy.
- User reports basic UI needs 10-20 revisions; root architecture is probably wrong.
- Package contains both old and new approaches at the same time.

13. Grandis Legacy Specific File Cleanliness Notes

- Do not keep repeated old/final/fix copies in active source.
- Document removed duplicates when cleanup is performed.
- For active Local AI source, public-facing version should be clean such as v3.20, not internal fix detail.
- Historical RC/starter packages may be saved offline but should not remain active source if superseded.
- Card definition locks, blueprint locks, CSV locks, UI design locks are authority packages; do not silently overwrite them.
- When moving toward PvP, use clean runtime/UI base rather than copying all historical packages into deploy.

Document status: Package Rules v1.1 - PDF lock update. Includes user requirements discussed during Grandis Legacy Local AI, Runtime Foundation, and PvP Railway work.

14. PDF Lock Document Rule

Current lock documents should be distributed as PDF unless the user explicitly asks for editable DOCX files.

- PDF is the active readable/reference format for package rules and runtime/game-flow rulebook documents.
- DOCX may remain an editing source or internal comparator, but should not be treated as the active published lock if a newer PDF exists.
- Do not include both old DOCX and new PDF lock documents in a final package unless the package manifest clearly documents why both are included.
- If a PDF lock replaces a DOCX source, package docs should point to the PDF as authority and mark the DOCX as source/archive only if included.

15. Runtime Documentation Packaging Rule

- Runtime packages should include only the current lock documents needed by the target package scope.
- For Local AI and PvP Railway packages, docs should separate implementation notes, release notes, QA reports, and lock references.
- Do not ship stale rule documents that conflict with the current runtime/card database direction.
- When rule wording changes are added, the changelog must identify whether they are wording-only, runtime-affecting, data-affecting, or package-only.

16. Grandis Legacy Attachment Wording Documentation Rule

Attachment and ongoing-effect wording should be kept in the current rulebook PDF and mirrored into runtime tests/data only when gameplay behavior changes.

Category	Standard wording
Attack attachment - 1 turn	This turn.
Attack attachment - 2 turns, exhausted-source case	Next turn.
Attack attachment - 2 turns, non-exhausted-source case	Until the end of your next turn.
Defense attachment - 1 turn	Until the start of your next turn.
Defense attachment - 2 turns	Until the start of your second turn.

17. Versioning Note for v1.1

- v1.1 is a documentation-format and lock-reference update, not a Local AI/PvP gameplay package.
- This update does not replace runtime executable tests or card database validation.
- Future packages should cite the PDF lock documents instead of old DOCX lock files when documenting rules/source-of-truth.